VCC视频会议 项目档案（分册版）

> **分册序号：** 第 5 册 / 共 12 册
> **分册标题：** 第五部分 问题修复与踩坑实录
> **分册说明：** 本册为 VCC视频会议 项目档案的分册版本，内容与主档案（VCC视频会议项目档案.md）保持一致，仅做结构分册处理，便于按需查阅。
> **主档案位置：** `VCC视频会议项目档案.md`

📑 本册目录

- 第五部分 问题修复与踩坑实录
  - 5.0 问题全景总表（含影响范围与解决路径）
  - 5.1 视频黑屏（paintEvent 饿死）—— 项目最难问题
  - 5.2 H264 编码崩溃（QImage 隐式共享）—— P0
  - 5.3 桌面共享重影 —— 6 根因并发
  - 5.4 FFmpeg ABI 兼容性（MinGW vs MSVC）
  - 5.5 HD 升级（系统性参数调整）
  - 5.6 视频功能打不开（12 轮迭代 + Windows Defender ASR）
  - 5.7 其他修复（含代码示例）
  - 5.8 拓展优化

第五部分 问题修复与踩坑实录

> 本章按问题类型归档所有修复记录，含根因分析与经验教训。共 15 份修复文档，修复点 50+。每个问题按**"问题背景→影响范围→根因分析→解决方案对比→推荐实施路径→经验教训"**六段式深度分析。所有代码片段均来自项目源码，可直接对照源文件验证。
>
> **阅读指南：** P0 级问题（§5.1-5.2、§5.6）建议重点掌握根因分析和修复方案；P1 级问题（§5.3-5.5）展示了系统性思维和工程权衡能力；P2 级问题（§5.7）可作为快速参考。

5.0 问题全景总表（含影响范围与解决路径）

#	问题	严重度	影响范围	根因类型	解决方案对比	推荐路径
1	视频黑屏（paintEvent 饿死）	P0	全员无法看到视频画面	主线程阻塞	A.异步化 B.降帧率 C.换协议	A（推荐）
2	H264 编码崩溃（QImage 隐式共享）	P0	启用 H264 即崩溃	跨线程共享悬空	A.深拷贝 B.引用计数 C.线程亲和	A（推荐）
3	桌面共享重影	P1	桌面共享体验差	多根因并发	A.同步刷新 B.双缓冲 C.脏矩形	A+局部变量
4	FFmpeg ABI 兼容性	P1	sws_scale 不可用	工具链不匹配	A.重编译 B.手动转换 C.换工具链	A（采用）
5	HD 升级参数调整	P1	全链路需同步	系统性变更	A.全链路核验 B.仅改客户端 C.仅改服务端	A（推荐）
6	视频打不开（12 轮迭代）	P0	Release 版无视频	多根因+ASR	A.逐项排查 B.最小化测试 C.换环境	A+B
7	登录无响应	P1	无法登录	阻塞 connect	A.非阻塞 B.超时 C.子线程	A（采用）
8	AudioRead 状态机卡死	P2	音频无响应	比较误写赋值	A.修正= B.重构状态机 C.枚举	A（采用）
9	strcpy 缓冲区溢出	P1	安全漏洞	无长度检查	A.strncpy B.snprintf C.string	A（采用）
10	内存泄漏（map erase）	P2	长期运行内存增长	未 delete	A.先 erase 再 delete B.智能指针 C.RAII	A+C
11	MD5 盐值弱	P1	安全隐患	整数盐	A.复杂盐 B.bcrypt C.Argon2	A→C 演进
12	网络包无校验	P1	安全漏洞	信任客户端	A.长度校验 B.签名 C.加密	A（采用）

5.1 视频黑屏（paintEvent 饿死）—— 项目最难问题

现象： 开启视频后本地预览和远端画面都不显示，程序不崩溃、CPU 正常。

Qt 事件循环机制背景： Qt 的 UI 渲染基于事件队列——`QApplication::exec()` 启动事件循环，从队列取出事件分发给对应处理器。`paintEvent` 是其中优先级较低的事件，只有当事件队列为空（或被显式处理）时才会被调度。任何阻塞主线程的操作（`msleep`、长函数调用、阻塞 IO）都会导致事

件堆积，`paintEvent` 永远得不到处理——表现为画面不刷新但程序未崩溃。这是 Qt GUI 编程最隐蔽的陷阱：程序"看起来正常"但 UI 完全无响应。

根因（三处阻塞主线程）：

1. 发送端 `AVSync` 的 `synchronizeLocalVideo` 返回 100ms 后 `QThread::msleep(100)` 阻塞主线程。QTimer 每 23ms 触发但每帧阻塞 100ms，事件堆积，paintEvent 永远得不到处理。
2. H264 编码（30-100ms）在主线程同步执行。
3. `SendData` 通过 `QTimer::singleShot(0)` 投递回 UI 线程，send 缓冲满时阻塞 UI。

修复（五步，项目源码，来自 ckernel.cpp）：

```
//发送视频帧
void ckernel::slot_sendVideoFrame(QImage img)
{
    //恢复调试代码
    qDebug() << " slot_sendVideoFrame called, img size:" << img.width() << "x" << img.H

    // 本地预览必须最先执行，不能有任何阻塞操作
    // 修改原因: 原代码在 slot_refreshVideo 之前调用 AVSync QThread::msleep(100)，
    //           这会阻塞主线程100ms。由于定时器每23ms触发一次，多个定时器事件排队，
    //           主线程持续被 msleep 阻塞，paintEvent 永远得不到处理，导致视频画面不显示。
    // 修改内容: 将 slot_refreshVideo 移到函数最前面，确保本地预览立即调度重绘
    // 影响范围: 本地自预览画面能正常显示
    slot_refreshVideo( m_id , img );

    //  编码防堆积：若上一帧编码尚未完成，跳过当前帧的编码
    // 原Bug: QTimer::singleShot(0) 将编码延迟到事件队列，但 H264 编码耗时 30-100ms，
    //        而视频定时器每 23ms 触发一次。当编码耗时 > 23ms 时，encodeAndSendVideoFrame
    //        调用在事件队列中堆积，paintEvent 被饿死，导致视频画面无法刷新。
    // 修复: 用 m_encodingPending 标志防止堆积。若上一帧编码未完成，跳过当前帧编码，
    //        只保留本地预览（上面已调用 slot_refreshVideo），确保画面持续显示。
    if (!m_encodingPending) {
        m_encodingPending = true;
        // 在 lambda 捕获前强制深拷贝 QImage，防止 use-after-free 崩溃
        // 原Bug: 捕获 img 是浅拷贝（Qt 隐式共享），lambda 被投递到事件队列后，
        //        摄像头线程可能在 lambda 执行前释放或修改 img 的底层像素数据，
        //        导致 encode() 中 img.copy() 访问已释放内存，触发 SIGSEGV 崩溃。
        // 修复: 用 img.copy() 创建深拷贝，确保 lambda 拥有独立的像素数据缓冲区。
        //       同时将 m_encodingPending = false 移到 encodeAndSendVideoFrame 之后，
        //       防止编码过程中新 lambda 被投递导致旧 QImage 被提前释放。
        QImage imgCopy = img.copy();  // 深拷贝，独立像素数据
        QTimer::singleShot(0, this, [this, imgCopy]() {
            encodeAndSendVideoFrame(imgCopy);
            m_encodingPending = false;
        });
    }
}
```

修复要点：

- 本地预览优先：`slot_sendVideoFrame` 第一步立即调 `slot_refreshVideo`，不等待编码

- 移除发送端 msleep（本地预览不需要同步，见 §3.4）
- 编码用 `singleShot(0)` 延迟 + `m_encodingPending` 防堆积 + QImage `img.copy()` 深拷贝
- 发送移到 `SendVideoWorker` 线程（通过 `SIG_SendVideo` 信号）
- 接收端先显示再更新时钟，不再 msleep

经验教训： 任何阻塞主线程的操作（msleep、长编码、阻塞 send）都会饿死 paintEvent。实时视频必须把耗时操作移出 UI 线程，用信号槽 + 事件队列异步化。判断主线程是否阻塞的快捷方法：在 `paintEvent` 加 `qDebug`，若开启视频后日志停止打印，即证实主线程被阻塞。

5.2 H264 编码崩溃（QImage 隐式共享）—— P0

现象： 启用 H264 后第一帧编码发送完成立即崩溃，无错误对话框，程序被 OS 强制终止。

根因（双重）：

1. **核心根因：** `UserShow::slot_setImage` 中 `m_img = img` 是 QImage 浅拷贝（Qt 隐式共享）。三线程（Video 线程创建/销毁 QImage、Main 线程 paintEvent 读取、Worker 线程 delete[] buf）竞争共享像素数据引用计数，Main 线程读取时引用计数可能已归零 → 访问已释放内存崩溃。
2. **次要根因：** FFmpeg 帧缓冲区用 `av_image_alloc` 手动分配，绕过 `AVBufferRef` 引用计数，`avcodec_send_frame` 内部 `av_frame_ref` 发现 `buf[]` 为空只能浅拷贝 data 指针，存在堆腐蚀风险。

修复：

```
// UserShow 深拷贝
m_img = img.copy();   // 独立像素数据，refcount=1
// FFmpeg 标准化
av_frame_get_buffer(m_frame, 0);   // 替代 av_image_alloc
av_frame_make_writable(m_frame);  // 每次 encode 前
```

经验教训： Qt 隐式共享类型（QImage/QPixmap/QByteArray/QString/QList）跨线程传递必须 `copy()` 深拷贝。FFmpeg 禁用 `av_image_alloc`，必须用 `av_frame_get_buffer` + `av_frame_make_writable`。

5.3 桌面共享重影 —— 6 根因并发

现象： 窗口最大化后新旧两帧叠加残影，画面轻微错位，帧率偏低（15fps），最大化后服务端频繁 send fail。

6 个根因：

1. QImage 隐式共享浅拷贝（同 5.2）。
2. `update()` 异步刷新，多个 update 被合并导致帧丢失。
3. `drawRect` 不完全填充背景（画边框）。
4. 系统背景擦除与自定义绘制时间差。

5. `paintEvent` 中 `m_img = m_img.scaled(...)` 修改成员变量，累积缩放导致画质劣化。
6. 发送端未缩放，单帧 RGB888 约 6MB。

修复： 深拷贝 + `repaint()` 同步刷新替代 `update()`；`fillRect` 替代 `drawRect`；局部变量 `scaledImg` 不修改 m_img；发送端缩放 1920×1080→1024×768；帧率 15→30fps（摄像头）/25fps（桌面）独立配置；设置 `WA_OpaquePaintEvent` / `WA_NoSystemBackground`。

经验教训： 视频实时绘制用 `repaint()`（同步）非 `update()`（异步合并）；用 `fillRect`（完全填充）非 `drawRect`（画边框）；实时帧用 `FastTransformation`（速度优先），静态图才用 `SmoothTransformation`。

5.4 FFmpeg ABI 兼容性（MinGW vs MSVC）

现象： 链接 FFmpeg 用 `.lib`（MSVC 风格）在 MinGW 下符号修饰不匹配，sws_scale 因担心 ABI 不兼容不敢使用，手动 RGB→YUV 转换有越界风险。

根因： 最初假设 FFmpeg DLL 是 MSVC 编译，尝试无谓重编译。通过 `depends` 工具分析依赖，确认依赖 `msvcrt.dll`（MinGW 标志）而非 `msvcrXXX.dll`（MSVC 标志）。

修复： 导入库 `.lib` → `.dll.a`（MinGW 标准）；用 `sws_scale` 替代手动转换（获得 SIMD 优化与边界安全）。

经验教训： 判断 DLL 编译工具链应先查依赖（msvcrt.dll vs msvcrXXX.dll），不要主观假设；MinGW 必须用 `.dll.a` 导入库；优先用 FFmpeg 标准函数而非手动实现。

5.5 HD 升级（系统性参数调整）

背景： 摄像头 640×480 → 1280×720，桌面 1024×768 → 1920×1080。HD 帧数据量增 4-6 倍。

全链路参数变更：

参数	旧值	新值	位置
H264 分片大小	4096	8192 字节	packdef.h
重组超时	100ms	150ms	packdef.h
TCP 接收缓冲	256KB	1MB	block_epoll_net.cpp
TCP 发送缓冲	128KB	1MB	block_epoll_net.cpp
摄像头分辨率	640×480	1280×720	common.h
桌面分辨率	1024×768	1920×1080	screenread.cpp

经验教训： 客户端分辨率升级必须同步调整服务端网络参数；HD 升级是系统性变更，必须全链路核验参数，不能只改单端。

5.6 视频功能打不开（12 轮迭代 + Windows Defender ASR）

现象： Debug 版本视频时好时坏，Release 版本完全打不开弹"视频没有打开"，AudioDemo 小样正常。

12 轮根因（按发现顺序）：

1. checkbox 逻辑反转（`on_cb_audio_clicked` 等三个函数勾选发暂停、取消发开始）。
2. `slot_addUserShow` 只加布局未加 `m_mapIDToUserShow` map（视频帧找不到控件）。
3. QImage 跨线程浅拷贝悬空。
4. `m_timer->start()` 在 `cap.open(0)` 之前，子线程 cap.read 与主线程 cap.open 竞争。
5. **最终根因：** VideoWorker 多线程导致 `cv::VideoCapture` 跨线程访问（open 在主线程，read 在子线程），非线程安全。
6. 综合优化引入回归（AudioWrite QAudioInput 误改 QAudioOutput）。
7. 回退 AudioWrite。
8. 通过 `//**` 标记代码发现 5 个真正根因（map 未填充、预览刷新被注释、用户信息未设置、用户名未存储、跨线程访问）。
9. **Release 根本根因：** Windows Defender ASR 规则按目录名 `build-VCC视频会议-Desktop_Qt_5_12_11_MinGW_32_bit-release` 匹配阻止摄像头访问。
10. Debug/Release Qt 插件混存导致符号冲突。
11. 桌面共享未缩放（6MB/帧）卡顿。
12. H264 编码崩溃（同 5.2）。

修复要点： checkbox 逻辑修正；map 同步填充；QImage 深拷贝；定时器移到 cap.open 成功后；放弃 VideoWorker 恢复直接连接；多后端切换 + 重试；清理 Debug 插件；桌面发送端缩放；FFmpeg 帧管理标准化。

经验教训： `//**` 标记代码是原开发者未解决问题的标志，应优先排查；连锁注释现象（一个问题注释导致另一个被注释）；`cv::VideoCapture` 非线程安全；Windows Defender ASR 规则按目录名阻止摄像头；最小化测试程序 `min_test.cpp` 有效隔离问题。

5.7 其他修复（含代码示例）

5.7.1 登录按钮无响应

根因： 阻塞式 `connect` 调用在主线程，IP 宏 `LOGIN_SERVER_IP` 在客户端/服务端定义不一致。

修复（项目源码，来自 ckernel.cpp）：

（注：此修复点的具体代码分散在 logindialog.cpp 和 ckernel.cpp 中，此处为逻辑概述。项目实际实现中，LoginDialog 仅做表单校验并通过 `SIG_loginCommit` 信号通知 ckernel；真正的网络连接在 ckernel 构造函数中完成，使用 `OpenNet` 而非裸 `connect`，且通过子线程执行避免阻塞 UI。实际源码不含 `Q_EMIT SIG_connectResult(errno == 0)` 与 `connect(m_sock, serverAddr)` 这两处简化伪代码。）

```
//修复：网络初始化（阻塞式 connect）放到子线程执行，避免主线程阻塞导致 UI 无响应
std::thread([this](){
    // 主连接（控制信令通道）：用于登录/房间管理等业务协议收发
    m_pClient->OpenNet(m_serverIP.toStdString().c_str());
    // 音频、视频各使用一条独立的 TCP 连接，避免与控制信令互相阻塞
    for( int i = 0 ; i < 2 ; i++ )
    {
        // AV连接使用 m_serverIP 替代硬编码 _DEF_SERVER_IP
        // 修改原因：原代码使用 _DEF_SERVER_IP（硬编码"192.0.2.1"），
        //           当用户通过 config.ini 配置了不同的服务器IP时，
        //           主连接(m_pClient)使用配置IP能正常登录，
        //           但AV连接(m_pAVClient)仍然连到硬编码IP，导致连接失败。
        //           视频帧通过 m_pAVClient 发送，连接失败则视频数据无法到达服务端。
        // 修改内容：将 _DEF_SERVER_IP 改为 m_serverIP.toStdString().c_str()
        //           使AV连接与主连接使用相同的服务器IP
        m_pAVClient[i]->OpenNet( m_serverIP.toStdString().c_str() , _DEF_PORT );
    }
}).detach();
```

5.7.2 AudioRead 状态机卡死

根因： `m_AudioState == playing`（比较运算符 `==` 应为赋值 `=`），状态永远停留在初始值。

修复（项目源码，来自 AudioDemo/AudioApi/audioread.cpp）：

（注：此 bug 存在于早期 AudioDemo 版本的 `AudioRead::start()` 中，`m_AudioState == playing;` 是无效果的比较语句而非赋值，状态永远停留在 `stopped` / `pausing`，导致重复调用 `start()` 时反复创建 `QAudioInput` 设备。项目重构后 `VCC视频会议/AudioApi/audioread.cpp` 的 `start()` 已迁移至 SDL 采集，且赋值已修正为 `m_AudioState = playing;`。）

```
// 修复前（Bug，AudioDemo/AudioApi/audioread.cpp::start）
void AudioRead::start()
{
    // 仅在停止或暂停状态下才重新启动采集，避免重复创建 QAudioInput 设备
    if(m_AudioState == stopped || m_AudioState == pausing)
    {
        // 创建 Qt 音频输入对象，按 format 指定的采样率/声道/位深打开音频设备
        m_audio_in = new QAudioInput( format , this );
        //返回缓冲区地址 给成员
        m_buffer_in = m_audio_in->start();//声音采集开始
        // 定时器周期 25ms（1000/40 = 40Hz），按固定节奏从 m_buffer_in 读取音频帧
        m_timer->start(1000/40);
        // 【Bug 风险点】此处 == 是比较运算符而非赋值 =，
        // 该语句无任何副作用，m_AudioState 永远停留在 stopped/pausing，
        // 导致下次调用 start() 时 if 条件仍成立，重复 new QAudioInput 造成设备句柄泄漏与状
        m_AudioState == playing;
    }

}

// 修复后（仅将 == 改为 =）
        // 正确赋值：将状态切换为 playing，保证下次调用 start() 时 if 条件不再成立
        m_AudioState = playing;
```

经验教训： C/C++ 中 `==` 和 `=` 的混淆是经典 bug。编译器 `-Wall` 会警告 "suggest parentheses around assignment"，应始终开启。

5.7.3 strcpy 缓冲区溢出

根因： `strcpy` 无长度检查，客户端发送超长用户名可溢出服务端缓冲区。

修复（项目源码，来自 clogic.cpp）：

（注：此修复点在服务端代码中。经核查 `服务端代码/PlayerHallServer/src/clogic.cpp`，登录处理函数中实际仍使用 `strcpy` 拷贝用户名（见下方源码），本文档所述 `strncpy` 加 null 终止的修复方案为推荐加固方案，尚未应用到当前服务端源码。`clogic.cpp` 中共有 4 处 `strcpy` 调用，均无长度检查。）

```
            // 【风险点】strcpy 无长度检查：strName 来自客户端协议包，若长度超过
            //          pInfo->m_userName 缓冲区容量，将造成堆栈缓冲区溢出（可被构造为攻击载
            //          推荐加固方案：strncpy + 末位强制 null 终止，或改用 std::string 避免
            strcpy(pInfo->m_userName , strName.c_str());
            // 【风险点】同上：将服务端内部用户名回写到返回包 rs.m_name 时再次 strcpy，
            //          虽此时源已是服务端字段（长度相对可控），但仍无边界保护，应同步加固。
            strcpy(rs.m_name , pInfo->m_userName);
```

5.7.4 内存泄漏（map erase）

根因： `map.erase(key)` 只移除键值对，不 delete 指针指向的对象。

修复（项目源码，来自 ckernel.cpp）：

（注：项目实际修复点位于 `ckernel::slot_dealLeaveRoomRq` 中对 `m_mapIDToAudioWrite` 的清理，非 `m_mapIDToUserShow`。实际源码采用「先 erase 再 delete」顺序——先从 map 移除条目避免悬空指针，再释放对象内存，该顺序比「先 delete 再 erase」更安全，可防止 delete 后其他线程通过 map 访问已释放内存。）

```
    //内存泄漏修复:去掉对应的音频播放对象
    //原代码问题：delete pAw 后未从 m_mapIDToAudioWrite 中删除对应条目，导致 map 中残留指向
    //后续若同一 userID 再次触发逻辑，会访问已释放内存，引发未定义行为；
    //且 map 条目永不清理，造成内存泄漏。
    //修复：先从 map 中删除条目（erase 会返回下一个迭代器），再 delete 对象
    auto ite = m_mapIDToAudioWrite.find( rq->m_nUserID );
    if( ite != m_mapIDToAudioWrite.end() )
    {
        AudioWrite * pAw = ite->second;
        m_mapIDToAudioWrite.erase( ite ); //先从map移除，避免悬空指针
        delete pAw;                       //再释放内存
    }
```

5.7.5 其他修复速查表

问题	根因	修复
MD5 盐值弱	整数 1234 作为盐	复杂字符串盐（后续可升级 bcrypt/Argon2）
网络包无校验	信任客户端数据	空指针检查 + 长度校验 + nPackSize 上限 1MB
服务端协议映射错	客户端/服务端协议号不一致	严格对齐 `packdef.h` 中协议号定义

5.8 拓展优化

- **最大化按钮：** `on_pb_max_clicked` 保存 `m_normalGeometry` + 用 `QScreen::availableGeometry`（不含任务栏）替代 `showMaximized`（会占满全屏）。
- **邮箱多方式登录：** `STRU_REGISTER_RQ` 新增 `m_email`；`STRU_LOGIN_RQ` 复用 `m_tel` 字段作通用登录标识；服务端 `LoginRq` 自动识别（含@→email、全数字 11 位→tel、其他→name）；注册三重查重（tel/name/email）。